MÓDULO PROFESIONAL

Desarrollo de Aplicaciones Móviles

Análisis del entorno y desarrollo de aplicaciones móviles

Investigación, análisis y configuración de un entorno de desarrollo.

Nombre	*Adrián Velasco Mañas*
Curso	*2º DAMP (Desarrollo de Aplicaciones Multiplataforma)*
Módulo	*Programación multimedia y dispositivos móviles*
Entrega	*04/10/2026*

Índice

1 Análisis de las limitaciones de los dispositivos móviles 2

1.1 Identificación de las principales limitaciones 2
1.2 Comparativa de 5 dispositivos móviles 6

2 Identificación de tecnologías de desarrollo móvil 12

2.1 Java (desarrollo nativo de Android) 12
2.2 Kotlin (desarrollo nativo de Android, moderno) 13
2.3 Flutter 13
2.4 React Native 14
2.5 .NET MAUI 14
2.6 Swift (desarrollo nativo de iOS) 15
2.7 Reflexión sobre la comparativa 16

3 Configuración del entorno de desarrollo Android 18

3.1 Requisitos e instalación de Android Studio 18
3.2 Creación de un proyecto de prueba 19
3.3 Comprobación del funcionamiento 20
3.4 Problemas frecuentes y cómo solucionarlos 22
3.5 Lista de evidencias pendientes de capturar 23

4 Identificación de configuraciones de dispositivos móviles 24

4.1 Resolución de pantalla y densidad 24
4.2 Sistema operativo y versión 25
4.3 Almacenamiento interno 26
4.4 Memoria RAM y potencia de procesamiento 27
4.5 Conectividad 28
4.6 Tabla resumen de los cinco criterios 29
4.7 Conclusión del apartado 4 30

5 Establecimiento de perfiles de usuario 31

5.1 Tabla de perfiles 31
5.2 Perfil de accesibilidad y diseño: los números concretos 32
5.3 Reflexión final: por qué perfiles y dispositivo van juntos 34

Conclusiones finales 35

Referencias bibliográficas 36

(ANTES DE NADA): Cómo está organizado este documento. Cada apartado termina con una caja de *fuentes* que indica qué páginas concretas se han usado para justificarlo (no es para decorar). Todas las páginas se han consultado el **1 de octubre de 2026**.

1 Análisis de las limitaciones de los dispositivos móviles

1.1 Identificación de las principales limitaciones

Un dispositivo móvil es un ordenador con tres restricciones que no aparecen en un portátil de sobremesa: **los recursos no son infinitos**, **la energía se agota** y **el usuario se lo lleva a todas partes** (y eso incluye ascensores, trenes y zonas sin cobertura). Las tres limitaciones que explico a continuación son las que más veces se repiten cuando hay un problema real.

Memoria RAM limitada

¿En qué consiste? Sabemos que la memoria RAM se utiliza para guardar datos temporales y procesos que se usan en el móvil, además de usarse para que el sistema operativo y las aplicaciones puedan funcionar correctamente, así que al desarrollar una aplicación móvil hay que saber gestionar la memoria para que el equipo pueda funcionar bien.

¿Por qué afecta a una aplicación? Una aplicación que use demasiada memoria puede hacer que el sistema se vea obligado a cerrar otras aplicaciones que estaban en segundo plano (o forzar el cierre de la aplicación).

Dato de contexto (muy relevante). Android lleva años publicando en su documentación oficial unas tablas de «comportamiento incorrecto» (*bad behavior*) que dependen de la gama de RAM del dispositivo. Para una app normal en primer plano, el límite va desde 2,00 GB en dispositivos de 4 GB de RAM hasta 4,25 GB en dispositivos de 16 GB o más. Es decir: **el mismo código consume el doble de margen en un móvil de gama alta que en uno de gama baja**, y el sistema lo mata antes de que tú te enteres.

Ejemplo práctico. Imaginemos una aplicación de edición de imágenes que carga archivos de alta resolución en la memoria. Si se utiliza en un dispositivo con poca RAM, la aplicación puede relentizar el sistema o incluso puede cerrarse porque el sistema se vea obligado a hacerlo.

¿Cómo se soluciona? La solución más evidente es optimizar el consumo de memoria pero, ¿cómo se hace? Normalmente, liberar objetos del código o recursos que no se utilizan pueden reducir el consumo, aunque hay más cosas que podemos hacer, como no cargar todas las imágenes al mismo tiempo (tendríamos que cargarlas poco a poco), además de reducir la resolución de estas. También podemos usar una técnica llamada lazy loading, que retrasa la carga de un recurso o fragmento de código hasta el momento en que se necesita.

Potencia de procesamiento limitada

¿En qué consiste? No todos los teléfonos tienen la misma capacidad de procesamiento ni todos gestionan de la misma manera los procesos asociados al rendimiento, al consumo ni a la temperatura. Un dispositivo con poca potencia de procesamiento puede tener dificultades para ejecutar operaciones dentro de nuestras aplicaciones, dificultades que en un dispositivo de alta gama no tendría.

¿Por qué afecta a una aplicación? Si dentro de nuestra aplicación se realizan demasiados cálculos, el dispositivo puede gastar muchos recursos del procesador para poder realizar estos cálculos, lo que provocaría lentitud en el móvil, poca fluidez (porque el procesador no podría ocuparse de todo al mismo tiempo y se emplearían menos recursos de lo normal en la interfaz gráfica), mucho tiempo de espera en operaciones (como abrir otras aplicaciones) y un mayor consumo de la batería (estresamos el procesador y este necesita alimentarse).

> **Lo que dice la plataforma.** Android ofrece dos APIs oficiales para saber con qué estás jugando: `ActivityManager.getMemoryClass()` devuelve, en megabytes, la memoria disponible para tu aplicación, y `ActivityManager.isLowRamDevice()` te dice directamente si estás en un dispositivo de gama baja. Y hay otra cosa interesante para el apartado 4: **Android 17 ha introducido límites de memoria por aplicación según la RAM total del dispositivo**, precisamente para evitar que una app descontrolada tumbe la interfaz del sistema entero.

Ejemplo práctico. Imaginemos que desarrollamos un juego 3D (con los renderizados y shaders de C++ y sus librerías) e intentamos renderizar la parte gráfica de forma constante: si tenemos un dispositivo potente no habrá problema, pero con un móvil de gama baja el juego podría presentar una bajada de FPS y podría reducirse la fluidez de forma considerable.

¿Cómo lo solucionamos? Entre otras muchas cosas (como monitorear el rendimiento en partes del mapa con alto renderizado) podemos optimizar los algoritmos de renderizado o funcionalidades con mucho cálculo (de hecho, una de las opciones que los desarrolladores de videojuegos móviles recomiendan es agrupar los objetos estáticos, para que el procesador o la gráfica pueda renderizarlos de una pasada, en vez de renderizar los objetos por separado). También podemos utilizar procesos en segundo plano para operaciones complejas, en vez de ejecutarlos en el hilo principal.

Consumo de batería

¿En qué consiste? Casi todos los componentes de un móvil (por no decir todos) consumen batería, sobre todo la pantalla, el procesador, la gráfica, el GPS y la cámara, por lo que una aplicación que use todo eso sin preocuparse por la batería tendrá un problema de consumo importante.

¿Por qué afecta a una aplicación? Una aplicación con un consumo elevado de batería puede provocar una pérdida de la vida útil de un móvil pero, ¿por qué? Básicamente los móviles tienen una salud de batería, que se calcula principalmente por los ciclos de carga del dispositivo y de la carga que se le aplique cuando se usa. Esto se traduce en que si nuestra aplicación gasta mucha batería de forma rápida, independientemente de si el móvil está cargando o no, estaríamos cargando mucho la batería, lo que perjudica a la batería del móvil (por ejemplo, si te quedas en videollamada durante varias noches con la pantalla encendida, verás como tu salud de la batería baja considerablemente, y es algo que no se puede revertir…).

Un detalle curioso. Los envíos por Bluetooth son todavía hoy el radio que más batería gasta del móvil, por encima del Wi-Fi. Por eso Android separa el Bluetooth clásico (transmitir audio, enviar archivos, conectar ratones) del Bluetooth Low Energy (LE), que está pensado justo para lo contrario: sensores, balizas y wearables que están casi todo el día conectados con un presupuesto de energía mínimo. Elegir el modo equivocado de Bluetooth puede costarte horas de autonomía.

Ejemplo práctico. Imaginemos (otra vez) una aplicación de navegación que para consultar la ubicación necesita tener la pantalla encendida y está constantemente usando el GPS para actualizar la ubicación al instante, descargando parte del mapa alrededor del usuario: sin optimizar estos procesos, la batería del móvil se podrá gastar durante algunas pocas horas de uso constante de la aplicación.

¿Cómo lo solucionamos? Podemos evitar procesos innecesarios (tanto en primer plano como en segundo plano, no queremos procesos innecesarios), además de reducir el tiempo que tarda el GPS en actualizar la ubicación (no necesitamos una precisión milimétrica constante). También podemos reducir el uso de sensores cuando no sean necesarios y detener la aplicación cuando no se está utilizando (si el usuario minimiza la aplicación, no sería coherente seguir pidiendo constantemente la ubicación al GPS, podemos actualizar la ubicación una vez el usuario vuelva a la app).

Conectividad a Internet limitada o inexistente

¿En qué consiste? Teniendo en cuenta que el móvil es un dispositivo móvil (valga la redundancia), es muy posible que nos encontremos casos donde la conexión no sea estable o no haya cobertura. Esto puede ser una limitación importante en nuestras aplicaciones, sobre todo si requerimos de Internet.

¿Por qué afecta a una aplicación? Si la aplicación necesita conectarse a Internet (o a algún servidor) con el objetivo de obtener recursos, ante una mala conexión podemos tener tiempos de espera elevados, archivos corruptos por la descarga a medias de un recurso, etc.

Ejemplo práctico. Imaginemos que nuestra aplicación necesita cada cierto tiempo actualizar y recargar recursos de la aplicación, recursos que tiene que descargar de un servidor: si nosotros usamos la aplicación en un metro, por ejemplo, y en algún momento nos quedamos sin conexión, podemos tener fallos de archivos corruptos o pantallas de carga infinitas por la mala conexión que tenemos.

¿Cómo lo solucionamos? Una buena práctica que podemos realizar para evitar este problema es implementar lógica y funcionamiento offline (al menos parcialmente), como guardar información importante en la caché, detectar cuando se pierde la conexión (y que no se quede la interfaz colgada esperando a que termine la operación). Otra cosa interesante que podemos hacer es separar los recursos en pequeños paquetes, de forma que si se interrumpe la conexión, está todo el recurso separado y cuando vuelva la conexión, se puede continuar con el siguiente paquete del recurso (así no tenemos archivos corruptos).

Y ojo con el 5G. La documentación oficial de Android para 5G nombra tres cosas que se supone que debes implementar en tu app y que casi nadie hace: comprobar si la red es de datos *tarificada* (`isActiveNetworkMetered()`, que evita que tu app descargue 200 MB de vídeo con los datos del usuario), detectar si la conexión es 5G, y estimar el ancho de banda del usuario. Esa última es la clave: no es que tu app vaya «a 5G», es que tiene que **adaptarse** a si el usuario va por Wi-Fi de casa o por datos en la calle.

Almacenamiento interno limitado

Esta limitación no aparece en la lista de «pistas» como si fuera la olvidada de la lista, pero en cuanto empiezas a usarla te das cuenta de que es de las que más falla. La verdad es que es más una limitación del **usuario** que del móvil, pero acaba afectando a la aplicación igual.

¿En qué consiste? El almacenamiento interno es el espacio donde vive todo: el sistema operativo, las aplicaciones instaladas, la caché y los datos que genera cada una. Es un recurso compartido entre todas las apps del dispositivo, y a diferencia de la RAM (que se limpia sola cuando cierras), **el almacenamiento solo crece** (salvo que el usuario borre cosas a mano).

¿Por qué afecta a una aplicación? Porque la aplicación compite con el resto. Si un usuario tiene un móvil de 64 GB con 12 GB libres y tu app necesita 300 MB, la instalación funciona, pero le estás quitando un 2,5 % de su espacio libre a alguien que probablemente tiene 3-4 fotos y un capítulo de serie descargado. Y aquí viene lo que no se ve venir: **no es solo la instalación**. Una app que guarda caché de imágenes sin límite acaba ocupando 1,5 GB sola, y entonces el usuario la desinstala.

Ejemplo práctico. Imaginemos una aplicación de podcast que descarga automáticamente los episodios nuevos para que el usuario pueda escucharlos sin conexión. El problema aparece con los datos móviles: la app descarga un episodio de 50 MB cada día, el usuario no se entera, y en dos meses tiene 3 GB de episodios que no ha escuchado y 1,2 GB de espacio que ha perdido sin saber por qué.

¿Cómo lo solucionamos? De varias formas a la vez. Lo primero, **darle al usuario el control**: una opción de «descargar solo por Wi-Fi» y otra de «borrar descargas» dentro de la app, que es lo que hace YouTube o Spotify y que evita el 90 % de las quejas. Después, **usar `MediaStore` en vez de almacenamiento interno** para los medios (fotos, audio, vídeo), que es la API oficial para archivos que el usuario quiere ver desde la galería y que no se pierden al desinstalar. Y por último, **cuidar la caché**: usar `getCacheDir()` para lo desechable y `getFilesDir()` solo para lo que de verdad necesitas guardar.

Diversidad de tamaños y resoluciones de pantalla

Esta la metía yo al final, aunque en el enunciado aparece en la lista de pistas, porque creo que es **la limitación que más veces ha roto las aplicaciones que he visto**, y no por culpa del móvil sino del desarrollador.

¿En qué consiste? No es que las pantallas sean «distintas», es que son de un montón de tipos a la vez. Hay teléfonos de 5” y de 7”, con relaciones de aspecto de 16:9 hasta 21:9, con densidades que van desde 160 dpi (mdpi) hasta 640 dpi (xxxhdpi), y hay tablets, relojes, pantallas plegables y televisores. Android no te pide que elijas una: te pide que funciones en todas.

¿Por qué afecta a una aplicación? Porque rompe la suposición más básica que tiene un desarrollador al empezar: **que un píxel es un píxel y que si la pantalla tiene 1440 px de ancho, mi contenido ocupa 1440 px**. No es así. Un icono de 48 dp mide 96 px en un móvil xhdpi y 192 px en uno xxxhdpi. Si dibujas todo a píxeles fijos, en unos dispositivos tu interfaz se verá diminuta y en otros se verá gigante y recortada.

Ejemplo práctico. Imaginemos una app de lista de la compra con las filas separadas por líneas de 1 píxel y los iconos de 24 dp. En un móvil de gama media con pantalla 720p se ve perfecta. Pero el usuario la instala en una tablet de 11 pulgadas con pantalla de 2560 px de ancho: las líneas de 1 píxel se vuelven invisibles, los iconos se ven diminutos al lado de un texto gigante, y toda la interfaz queda concentrada en una franja de 300 dp a la izquierda con medio mapa de la compra en blanco a la derecha.

¿Cómo lo solucionamos? Con dos reglas que parecen sencillas pero mucha gente no aplica. La primera: **nunca uses px para el layout**; usa `dp` para tamaños y `sp` para el texto (porque `sp` es el único que respeta el tamaño de letra que haya elegido el usuario en los ajustes de accesibilidad). La segunda, y más moderna: **no decidas el layout en función del hardware sino del espacio disponible**. Y un truco extra muy útil: usa **drawables vectoriales** en `res/drawable/` en lugar de PNG en `drawable-hdpi/`, `drawable-xhdpi/`, etc.

Fuentes del apartado 1.1: Android vitals · ActivityManager · Storage overview · Bluetooth overview · Develop for 5G · Screen densities · Screen sizes · Behavior changes Android 17

1.2 Comparativa de 5 dispositivos móviles

He elegido cinco teléfonos que sean **intencionadamente distintos entre sí**: tres de gama alta Android de fabricantes diferentes, un iPhone de gama alta (que además es el único iOS de la lista, para poder ver las diferencias de plataforma) y un Motorola de gama media-baja, que es el que más se acerca a lo que usa mucha gente que no se compra un móvil cada dos años.

Tabla 1.2.1 — Comparativa general

Cuadro 1: Comparativa general de los cinco dispositivos

Dispositivo	Procesamiento	Tamaño	Autonomía	Conectividad	Fuente
Samsung Galaxy S26 Ultra	Snapdragon 8 Elite Gen 5, 2×4,74 + 6×3,62 GHz · 12-16 GB RAM · 256 GB-1 TB	6,9" · 1440×3120 · Dynamic AMOLED 2X · 163,6×78,1×7,9 mm · 214 g	5.000 mAh · 60 W · hasta 31 h de vídeo	5G, Wi-Fi 7, BT 6.0, NFC, UWB, USB-C 3.2, satélite	Samsung
Google Pixel 10 Pro XL	Tensor G5, 1×3,78 + 5×3,05 + 2×2,25 GHz · 16 GB RAM · 256 GB-1 TB	6,8" · 1344×2992 · LTPO OLED · 162,8×76,6×8,5 mm · 232 g	5.200 mAh · 45 W · más de 24 h	5G, Wi-Fi 7, BT 6.0, NFC, UWB, Thread, USB-C 3.2, satélite	Google
OnePlus 13	Snapdragon 8 Elite, 2×4,32 + 6×3,53 GHz · 12-16 GB RAM · 256-512 GB	6,82" · 1440×3168 · LTPO ProXDR 120 Hz · 162,9×76,5×8,5 mm · 213 g	6.000 mAh · 80 W (100 W opcional) · sin cifra oficial	5G, Wi-Fi 7, BT 5,4, NFC, mando IR, USB 3.2	OnePlus

Dispositivo	Procesamiento	Tamaño	Autonomía	Conectividad	Fuente
iPhone 17 Pro Max	A19 Pro, 6 núcleos (2 rendimiento + 4 eficiencia) · 12 GB RAM · 256 GB-2 TB	6,9" · 1320×2868 · Super Retina XDR OLED · 163,4×78×8,75 mm · 233 g	∼5.088 mAh (no publicado) · MagSafe 25 W · hasta 39 h de vídeo	5G, Wi-Fi 7, BT 6, NFC, UWB, Thread, USB-C 10 Gb/s, satélite; solo eSIM en EE.UU.	Apple
Motorola Moto G47	MediaTek Dimensity 6300, 8 núcleos a 2,4 GHz · 8 GB RAM (+16 GB virtual) · 128 GB + microSD 1 TB	6,67" · 720×1604 · LCD · 166,2×76,5×7,85 mm · 191 g	5.200 mAh · 20 W · hasta 43 h	5G, Wi-Fi 5, BT 5,4, NFC, jack 3,5 mm, USB-C 2.0	Motorola

⚠ **Aviso sobre los datos.** Hay dos cosas que conviene decir con claridad. La primera es que **Apple no publica ni los mAh de la batería ni los GB de RAM del iPhone 17 Pro Max** (esas cifras vienen de terceros y las he marcado como tales). Y la segunda es que el Moto G47 tiene un **«RAM Boost» que no es RAM real**: Motorola utiliza parte del almacenamiento interno como memoria virtual para anunciar hasta 24 GB. Sirve para algo, pero no es lo mismo que 24 GB de RAM, y confundir esas dos cosas es un error muy habitual.

Tabla 1.2.2 — A. Capacidad de procesamiento

Cuadro 2: Comparativa de capacidad de procesamiento

	Samsung S26 Ultra	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Procesador / SoC	Qualcomm Snapdragon 8 Elite Gen 5 for Galaxy (TSMC 3 nm)	Google Tensor G5 (TSMC 3 nm) + coprocesador Titan M2	Qualcomm Snapdragon 8 Elite (TSMC 3 nm), módem X80	Apple A19 Pro (TSMC 3 nm, según terceros)	MediaTek Dimensity 6300 (6 nm)
Núcleos	Octa-core: 2× Oryon V3 a 4,74 GHz + 6× Oryon V3 a 3,62 GHz	Octa-core: 1× Cortex-X4 3,78 GHz + 5× Cortex-A725 3,05 GHz + 2× A520 2,25 GHz	Octa-core: 2× Oryon a 4,32 GHz + 6× Oryon a 3,53 GHz	Hexa-core: 2 rendimiento + 4 eficiencia	Octa-core: 2× A76 2,4 GHz + 6× A55 2,0 GHz
GPU	Adreno 840	PowerVR DXT-48-1536	Adreno 830	6 núcleos con ray tracing	Arm Mali-G57 M2C

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
RAM	12 GB (256/512 GB) · 16 GB (1 TB)	16 GB en todas las versiones	12 / 16 GB LPDDR5X	12 GB (no publicado)	8 GB LPDDR4X (+16 GB virtuales)
Almacenamiento	256 GB / 512 GB / 1 TB (UFS 4.x)	256 GB / 512 GB / 1 TB (UFS 4.0)	256 GB / 512 GB (UFS 4.0)	256 GB / 512 GB / 1 TB / 2 TB	128 GB + microSD hasta 1 TB
Ampliable	No	No	No	No	**Sí (único de los cinco)**

Tabla 1.2.3 — B. Tamaño

Cuadro 3: Comparativa de tamaño y pantalla

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Pulgadas	6,9”	6,8”	6,82”	6,9”	6,67”
Resolución	1440×3120 (QHD+)	1344×2992 (QHD+)	1440×3168 (QHD+)	1320×2868	**720×1604 (HD+)**
Densidad aprox.	∼500 ppi	∼486 ppi	∼510 ppi	460 ppi	**264 ppi**
Tecnología	Dynamic AMOLED 2X (LTPO), 2.600 nits, Gorilla Armor 2	LTPO OLED, 3.300 nits, Victus 2	ProXDR LTPO 4.1, 4.500 nits, 10 bits	Super Retina XDR OLED, 3.000 nits	**LCD de 8 bits**, 1.050 nits
Refresco	120 Hz adaptativo	1-120 Hz	1-120 Hz dinámico	ProMotion hasta 120 Hz	120 Hz
Dimensiones	163,6×78,1×7,' mm	162,8×76,6×8,: mm	162,9×76,5×8,. mm	163,4×78,0×8,' mm	166,2×76,5×7,85 mm
Peso	214 g	232 g	213 g	**233 g (el más pesado)**	**191 g (el más ligero)**

Fíjate en una cosa reveladora. La diferencia entre el mejor y el peor procesador no es «un poco más rápido», es **casi el doble de frecuencia de reloj** (4,74 GHz frente a 2,4 GHz) y, sobre todo, cambia de arquitectura de CPU (Oryon de 3 nm frente a Cortex-A76 de 6 nm). Pero la diferencia entre la mejor y la peor pantalla no está en los píxeles, está en el **brillo y en los bits de color**: el Motorola es un LCD de 8 bits, y el Samsung y el OnePlus son OLED de 10 bits con miles de nits.

Tabla 1.2.4 — C. Autonomía

Cuadro 4: Comparativa de autonomía y carga

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Batería	5.000 mAh	5.200 mAh	**6.000 mAh (el más grande)**	~5.088 mAh (terceros)	5.200 mAh
Carga con cable	60 W (75 % en 30 min)	45 W (70 % en 30 min)	**80 W, hasta 100 W con adaptador**	50 % en 20 min con 40 W	20 W
Carga inalámbrica	25 W	25 W (con imanes)	**50 W**	MagSafe 25 W / Qi2 25 W	**No tiene**
Declaración	Hasta **31 h** de vídeo	Más de **24 h**	**No publica horas**	Hasta **39 h** de vídeo	Hasta **43 h** (varía)
Extra	Carga inversa 4,5 W	Carga inversa 10 W	Carga inversa 10 W	Sin cargador en la caja	Carga inversa 6 W

Tabla 1.2.5 — D. Conectividad

Cuadro 5: Comparativa de conectividad

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
Redes móviles	2G/3G/4G/5G (SA y NSA) + mmWave EE.UU.	2G/3G/4G/5G (SA y NSA)	2G/3G/4G/5G (solo Sub-6)	5G sub-6 + mmWave, LTE gigabit	2G/3G/4G/5G (solo Sub-6)
Wi-Fi	Wi-Fi 7 (802.11be)	Wi-Fi 7 (802.11be)	Wi-Fi 7 (802.11be)	Wi-Fi 7 (802.11be)	**Wi-Fi 5 (802.11ac)**
Bluetooth	**6.0**	**6.0**	5,4	**6**	5,4
NFC	Sí (con UWB)	Sí	Sí (con eSE)	Sí (modo lector)	Sí (según mercado)
UWB	Sí	Sí	**No**	Sí (chip 2.ª gen.)	**No**
Otros detalles	Satélite (SOS y mensajes)	Satélite SOS, Thread/Matter	**Mando IR**, BeaconLink	Satélite, Thread, USB-C 10 Gb/s	**Jack 3,5 mm**, IP64, MIL-STD-810H

	Samsung	Pixel 10 Pro XL	OnePlus 13	iPhone 17 Pro Max	Moto G47
SIM	Nano-SIM + eSIM	Nano-SIM + eSIM	Doble nano-SIM	**Solo eSIM EE.UU.**, nano+eSIM Europa	2× nano-SIM, **sin eSIM**
USB	USB-C 3.2	USB-C 3.2	USB 3.2	USB-C hasta 10 Gb/s	USB-C 2.0

Fuentes del apartado 1.2: Samsung Galaxy S26 Ultra · Google Pixel 10 Pro XL · OnePlus 13 · iPhone 17 Pro Max · Motorola Moto G47 · Secundaria: GSMArena

Conclusión de la comparativa

¿Qué dispositivo presenta una mayor capacidad de procesamiento?

Depende de a qué le llamemos «mayor», y esa ambigüedad es instructiva. Si hablamos de **frecuencia de reloj y arquitectura**, el **Samsung Galaxy S26 Ultra** gana claramente: sus 2 núcleos Oryon V3 a 4,74 GHz son la CPU más rápida de las cinco, con una arquitectura de 3 nm. Si hablamos de **cantidad de RAM**, gana el **Pixel 10 Pro XL** con 16 GB en todas sus versiones. Y si hablamos de **memoria extra disponible para la GPU**, el iPhone 17 Pro Max tiene su chip A19 Pro con GPU de 6 núcleos y ray tracing por hardware. Lo que sí es un hecho es que **el Motorola Moto G47 está en otro planeta**: 2,4 GHz y 6 nm, es un chip diseñado para que la batería dure, no para que la app vaya rápida.

¿Cuál tiene una pantalla de mayor tamaño?

El Samsung Galaxy S26 Ultra y el iPhone 17 Pro Max empatan en 6,9 pulgadas, y por centímetros son prácticamente idénticos (163,6 mm y 163,4 mm de alto). El OnePlus 13 va inmediatamente detrás con 6,82", el Pixel 10 Pro XL con 6,8" y el Motorola Moto G47 se queda en 6,67". Pero el dato interesante no es la diagonal, sino que **la diferencia en calidad de pantalla sí es enorme**: el Motorola tiene 264 ppi en un LCD de 8 bits, y el OnePlus tiene 510 ppi en un OLED de 10 bits con 4.500 nits.

¿Qué diferencias encuentras en la capacidad de las baterías?

Tres diferencias de tipo distinto. La primera es la de **capacidad**: el OnePlus 13 gana con 6.000 mAh (silicio-carbono), seguido del Pixel y el Motorola (5.200 mAh), el iPhone (∼5.088) y el Samsung (5.000 mAh). La segunda es la de **velocidad de carga**, que es donde se ve una brecha enorme: de los 20 W del Motorola a los 80 W del OnePlus (100 W si compras su cargador). Y la tercera, la que más me sorprendió, es la de **las declaraciones del fabricante: no son comparables entre sí**. Samsung da 31 horas de vídeo, Apple 39 horas, Motorola 43 horas sin especificar y OnePlus no publica ninguna cifra.

¿Qué dispositivos ofrecen más opciones de conectividad?

Por orden, el **Samsung Galaxy S26 Ultra** es el más completo: 5G con mmWave, Wi-Fi 7, Bluetooth 6.0, NFC, UWB, USB 3.2, eSIM y satélite. Le siguen muy de cerca el **iPhone 17 Pro Max** y el **Pixel 10 Pro XL**. El **OnePlus 13** es el más curioso: tiene mando IR pero se queda **sin UWB**. Y el **Motorola Moto G47** es el que más se aleja: Wi-Fi 5 en lugar de Wi-Fi 7, sin UWB, sin eSIM, sin satélite y USB 2.0.

¿Cómo podrían influir estas diferencias en la ejecución de una misma aplicación móvil?

Esta es la pregunta importante, y la respuesta corta es: **mucho**. Si ejecutamos la misma app en los

cinco dispositivos, lo que cambia no es que «vaya más o menos rápido» de forma vaga, sino que hay funcionalidades enteras que existen en unos y directamente no existen en otros. Una app de pago contactless que use NFC funciona en cuatro de cinco. Una app que aproveche el 5G para precargar vídeo tiene que tener un plan B para el Motorola. Una app con muchos efectos gráficos irá perfecta en el Samsung y «a trompicones» en el Moto G47. **Esa última paradoja es la mejor lección de toda la comparativa: más espacio y más potencia no van de la mano, y los cinco teléfonos combinan de la mejor manera posible los mismos cuatro recursos.**

2 Identificación de tecnologías de desarrollo móvil

Elegir tecnología es, sin exagerar, una de las decisiones más importantes del proyecto y también de las más caras de revertir: cambiar de framework a mitad de una app es casi como empezar otra vez. Por eso en este apartado no solo listo las opciones, sino que intento explicar **por qué** cada una existe y **en qué situación concreta** sería la mejor elección.

He investigado seis tecnologías: las tres que son soluciones de facto para Android (Java, Kotlin y el framework de interfaz Jetpack Compose como contexto), y las tres multiplataformas o nativas de iOS más usadas (Flutter, React Native, .NET MAUI y Swift). Son más de cuatro, que es el mínimo que pide el enunciado, y así puedo comparar de verdad en lugar de listar sin criterio.

> **Aclaración de partida.** Java y Kotlin no son «competidoras» la una de la otra, porque en la práctica **se usan juntas**. Java es el lenguaje histórico de Android y hay millones de líneas de código ahí; Kotlin es el que Google recomienda para todo lo nuevo y está pensado para convivir con el Java antiguo. Lo que sí son alternativas reales entre sí son Kotlin, Flutter, React Native y MAUI.

2.1 Java (desarrollo nativo de Android)

Nombre de la tecnología. Java, el lenguaje original y oficial del desarrollo Android.

Lenguaje. Java (y Kotlin, que convive con él).

Tipo de desarrollo. **Nativo**. El código compila a bytecode y se ejecuta en la máquina virtual de Android (ART), sobre las APIs del sistema.

Sistemas operativos compatibles. Android. Y aquí está el detalle importante: **solo Android**. Si compilas con Java para Android, no te sirve para nada en iOS.

Principales características.

- Acceso completo y directo a todas las APIs del sistema (cámara, GPS, Bluetooth, sensores, notificaciones, Wear OS…).
- Interfaz de dos formas: con **vistas XML** (`res/layout/*.xml`) o de forma **declarativa con Jetpack Compose**.
- Herramienta oficial: **Android Studio**, que incluye compilador (Gradle), emulador, inspector de memoria y de red, y un perfilador de CPU.
- **Dato relevante:** en mayo de 2026 Google publicó una entrada de blog titulada literalmente «Android UI development is Compose-first». El sistema de vistas clásico entra en **modo mantenimiento**.

Ventajas. Rendimiento máximo (código nativo, sin capas intermedias); cobertura total de la plataforma; ecosistema maduro; estabilidad a largo plazo.

Desventajas. Solo sirve para Android; mucho código repetitivo con el sistema de vistas; fragmentación (miles de fabricantes); mantenimiento costoso.

Ejemplo o situación adecuada. Una app de banca o de salud que necesite leer el NFC de la tarjeta

sanitaria, acceder a la cámara para escanear recetas y funcionar en el *longtail* de dispositivos Android baratos, donde el rendimiento y el tamaño del APK importan mucho.

2.2 Kotlin (desarrollo nativo de Android, moderno)

Nombre de la tecnología. Kotlin, el lenguaje oficial preferido por Google para Android.

Lenguaje. Kotlin.

Tipo de desarrollo. **Nativo** para Android, y además **multiplataforma** mediante **Kotlin Multiplatform (KMP)**.

Sistemas operativos compatibles. Android de forma nativa; y mediante KMP, la misma lógica de negocio se puede compartir con **iOS, macOS, Windows, Linux, web (Wasm/JS), tvOS y watchOS**.

Principales características.

- **Conciso y seguro**: el tipado nulo es del lenguaje, lo que elimina una enorme cantidad de errores en tiempo de ejecución.
- **Coroutines**: la forma oficial de hacer asincronía sin bloquear la interfaz.
- **Interoperabilidad total con Java**: puedes llamar a Java desde Kotlin y al revés.
- **Jetpack Compose**: la interfaz moderna declarativa. Y también **Compose Multiplatform**.
- Google mantiene una estrategia **Kotlin-first**: la documentación nueva sale primero en Kotlin.

Ventajas. Menos código y menos errores que Java; coroutines resuelven elegantemente el trabajo en segundo plano; KMP permite reutilizar la lógica entre Android e iOS; herramientas excelentes.

Desventajas. Curva de aprendizaje; KMP todavía madurando; interoperabilidad con el ecosistema antiguo; dependencia de Google.

Ejemplo o situación adecuada. Una app de comercio electrónico que nace para Android, tiene una lógica de precios, stock y carrito complicada, y en un año quiere publicar en iOS. Con KMP se comparte el 70-80 % del código de negocio.

2.3 Flutter

Nombre de la tecnología. Flutter, el framework multiplataforma de Google.

Lenguaje. **Dart**.

Tipo de desarrollo. **Multiplataforma** (una única base de código).

Sistemas operativos compatibles. Android (API 24+), iOS/iPadOS (15.0+), Windows, macOS, Linux y **web**. Es la lista más amplia de las seis tecnologías.

Principales características.

- **Flutter no usa los botones ni las vistas nativas**: dibuja todos los píxeles él mismo a través de su propio árbol de widgets.
- **Impeller** es su renderizador por defecto desde la 3.27 en iOS y en Android a partir de la API 29.

- **Hot reload** de verdad: cambias un `Text` y lo ves en el móvil en menos de un segundo.
- Ecosistema **pub.dev** con más de 1.300 millones de descargas en los últimos 30 días.
- Versión actual: **Flutter 3.47** (agosto de 2026).

Ventajas. Una sola base de código para todo; interfaz visual idéntica en todas partes; rendimiento muy cercano al nativo; hot reload excelente; consistencia.

Desventajas. Tamaño de la aplicación más grande; dependencia de plugins; el aspecto nativo no es exacto; Dart es un lenguaje «nuevo»; riesgo de plataforma única.

Ejemplo o situación adecuada. Una startup que quiere lanzar la misma app en iOS, Android, web y escritorio en tres meses con un equipo pequeño.

2.4 React Native

Nombre de la tecnología. React Native, el framework multiplataforma de Meta.
Lenguaje. **JavaScript o TypeScript**.
Tipo de desarrollo. **Multiplataforma** (Android e iOS principalmente).
Sistemas operativos compatibles. Android, iOS y, mediante proyectos externos (*out-of-tree*), macOS, Windows, visionOS, tvOS, web y HarmonyOS.

Principales características.

- A diferencia de Flutter, **sí usa las vistas nativas de cada sistema**.
- La **Nueva Arquitectura** (Fabric y Turbo Native Modules) es el modo por defecto desde la 0.76, y desde la **0.82 (octubre de 2025) es la única arquitectura**.
- **Fast Refresh** activado por defecto.
- Versión actual: **React Native 0.87** (agosto de 2026), que exige Node 22+.
- Ecosistema npm: cualquier librería de JavaScript probablemente ya tiene bindings.

Ventajas. Enorme mercado de desarrolladores; ecosistema npm gigantesco; integración incremental; buen rendimiento con la Nueva Arquitectura; plataformas adicionales.

Desventajas. Los módulos nativos son fuente de problemas; las actualizaciones mayores duelen; rendimiento variable; depurar es más complejo; menos presencia que Flutter en el mercado de frameworks.

Ejemplo o situación adecuada. Una empresa que ya tiene un equipo web grande y quiere lanzar la app móvil **sin contratar un equipo nuevo**.

2.5 .NET MAUI

Nombre de la tecnología. .NET MAUI (*Multi-platform App UI*), el sucesor de Xamarin.Forms dentro del ecosistema .NET de Microsoft.
Lenguaje. **C#**.

Tipo de desarrollo. **Multiplataforma**.
Sistemas operativos compatibles. Android (API 24+), iOS 11+, macOS (Mac Catalyst), Windows 10 1809+ con WinUI 3, y tvOS.

Principales características.

- **Usa los controles nativos de cada plataforma** mediante un modelo de *handlers*.
- **XAML** para declarar la interfaz y C# para la lógica.
- **Hot Reload**, incluido el XAML Hot Reload.
- Estado actual: **.NET MAUI 10** (noviembre de 2025). **.NET MAUI 9 dejó de soportarse en mayo de 2026 y .NET MAUI 10 termina su soporte en mayo de 2027**.

Ventajas. Un solo lenguaje y un solo IDE (Visual Studio); controles nativos reales; buena integración con .NET; cubre escritorio; C# es muy productivo.

Desventajas. Las críticas de la comunidad sobre estabilidad y fiabilidad son serias y están documentadas; ecosistema de UI más pequeño; rendimiento irregular en apps grandes; ciclo de soporte más corto (12 meses por versión); comunidad más pequeña.

Ejemplo o situación adecuada. Una empresa mediana con un equipo de C#/.NET que tiene una aplicación de escritorio Windows y quiere lanzar la versión móvil y la versión macOS con el mismo equipo.

2.6 Swift (desarrollo nativo de iOS)

Nombre de la tecnología. Swift, el lenguaje oficial de Apple.
Lenguaje. Swift.
Tipo de desarrollo. **Nativo** (exclusivamente del ecosistema Apple).
Sistemas operativos compatibles. iOS, iPadOS, macOS, tvOS, visionOS y watchOS. **No hay versión para Android**.

Principales características.

- **Dos formas de hacer interfaz**: **SwiftUI** (declarativo) y **UIKit** (imperativo). Apple dice que **conviven** y que se adopte «a tu propio ritmo».
- **Concurrencia moderna**: la versión actual es **Swift 6.4** (septiembre de 2026), con *strict concurrency*.
- **Xcode Previews**: ver la pantalla en el canvas lateral sin arrancar la app.
- **Integración profunda**: Core Data, SwiftUI, Metal, Core ML, HealthKit, ARKit…
- Seguridad de memoria gestionada (ARC).

Ventajas. Máximo rendimiento y fidelidad en plataforma Apple; acceso completo a cada API; lenguaje moderno y seguro; herramientas excelentes (Xcode + SwiftUI Previews); soporte a largo plazo coherente.

Desventajas. Lock-in total; coste de aprender dos cosas (SwiftUI + UIKit); ecosistema de herramientas más estrecho; evolución rápida de la API.

Ejemplo o situación adecuada. Cualquier app de calidad o de prestigio en el ecosistema Apple. Siri, Things, Fantastical, Bear, Overcast… todas están escritas en Swift.

Tabla 2.1 — Comparativa de las seis tecnologías

Cuadro 6: Comparativa técnica de las seis tecnologías

	Java	Kotlin	Flutter	React Native	.NET MAUI	Swift
Lenguaje	Java	Kotlin	Dart	JS/TS	C#	Swift
Tipo	Nativo	Nativo (+KMP)	Multiplataforma	Multiplataforma	Multiplataforma	Nativo
Android	Sí	Sí	Sí	Sí	Sí	**No**
iOS	**No**	Con KMP	Sí	Sí	Sí	Sí
Escritorio	**No**	Con KMP	Win/mac/Linux	Proyectos externos	Win/mac nativo	macOS
Web	**No**	Experimental	Sí	Proyectos externos	**No**	**No**
Cómo dibuja la UI	Vistas XML o Compose	Compose o XML	Motor propio	Vistas nativas	Controles nativos	SwiftUI o UIKit
Rendimiento	Máximo	Máximo	Muy alto	Alto	Medio-alto	Máximo
Tamaño de la app	Bajo	Bajo	**Alto**	Medio	Medio-alto	Bajo
Hot reload	Limitado	Sí (Compose)	Excelente	Fast Refresh	Sí (XAML)	Previews Xcode
Curva de aprendizaje	Alta	Media	Media	Media (web)	Media-alta	Media
Comunidad	Muy grande	Muy grande	Grande	Muy grande	Media-pequeña	Grande
Coste de cambiar	Medio	Medio	Alto	Alto	Alto	Medio

Fuentes del apartado 2: Kotlin for Android · Kotlin releases · Compose-first (19/05/2026) · Flutter platforms · Impeller · Flutter 3.47 · React Native 0.82 · .NET MAUI · Swift 6.4 · SwiftUI · Bitrise State of Mobile 2026 · Stack Overflow Survey 2024

2.7 Reflexión sobre la comparativa

¿Qué diferencias existen entre el desarrollo nativo y el multiplataforma?

La diferencia de fondo no es técnica, es **de dónde viene el poder**. En el nativo, la aplicación usa directamente los componentes del sistema. En el multiplataforma, hay **una capa intermedia** que traduce. Y de esa capa salen todas las diferencias:

- **El aspecto visual.** El nativo se ve nativo porque lo es.
- **El rendimiento y el consumo.** El nativo es más eficiente, siempre.

- **El acceso al hardware.** El nativo llega a todo. El multiplataforma a lo que alguien haya escrito un binding.
- **El tamaño de la aplicación.** El nativo pesa menos.
- **El coste de desarrollo.** Aquí el multiplataforma gana sin discusión, pero con una condición: si luego necesitas algo muy nativo en una de las dos, la ventaja se evapora.

Mi conclusión honesta, después de comparar las seis, es que **la decisión no es técnica, es organizativa**. Es la pregunta «¿qué equipo tengo y qué necesitan mis usuarios?».

¿Qué ventajas puede ofrecer una aplicación desarrollada específicamente para Android?

Cuatro, y las cuatro son reales:

1. **Acceso completo a todas las APIs de Android sin intermediarios.**
2. **Rendimiento y consumo óptimos.**
3. **Tamaño de descarga menor.**
4. **Publicación inmediata en un solo sitio.**

¿Qué beneficios puede aportar una tecnología que permita desarrollar para varios sistemas operativos?

1. **Coste.** El ahorro típico está entre el 30 % y el 50 % del coste total de desarrollo (no el 80 %, que es un mito).
2. **Velocidad de salida al mercado.**
3. **Un único equipo de mantenimiento.**
4. **Consistencia del producto y de los valores.**
5. **Actualizaciones simultáneas.**

¿Qué factores debería tener en cuenta una empresa antes de elegir una tecnología?

1. **¿Dónde están nuestros usuarios y qué necesitan?**
2. **El equipo: ¿qué sabe hacer y qué quiere aprender?**
3. **Requisitos técnicos muy específicos.**
4. **Presupuesto y plazo.**
5. **Mantenimiento a 5 años vista.**
6. **Riesgo de proveedor.**
7. **Accesibilidad y regulación.**

> **Si tuviera que dar un consejo concreto.** Empieza nativo con Kotlin y Compose si el negocio es Android, y usa KMP para compartir la lógica cuando aparezca el iOS. Empieza con Flutter si tienes un equipo pequeño y la app es visual. Y evita .NET MAUI salvo que ya seas una casa de C#, porque su ciclo de soporte corto y las críticas de estabilidad son un riesgo que no compensa en un proyecto nuevo.

3 Configuración del entorno de desarrollo Android

⚠ **Este apartado lo tienes que hacer tú.** Instalar Android Studio, crear el proyecto, ejecutarlo en el emulador y hacer las capturas **no se pueden hacer desde un documento**. Lo que te dejo aquí es la guía paso a paso con los valores exactos que hay que elegir, el motivo de cada decisión y los sitios donde hacer cada captura. Al final del apartado tienes la lista completa de evidencias que te faltan.

3.1 Requisitos e instalación de Android Studio

Paso 0 — Comprobar que tu ordenador cumple los requisitos.

Antes de descargar nada, hay que mirar los requisitos que publica Google. En mi caso el punto crítico siempre es el mismo: **la virtualización por hardware**.

Requisito	Windows (mínimo)	Windows (recomendado)
Sistema operativo	Windows 10 de 64 bits	Última versión de Windows de 64 bits
RAM	8 GB (solo Studio) · **16 GB (Studio + emulador)**	**32 GB**
Procesador	Con virtualización activada (Intel VT-x o AMD-V) en la BIOS; microarquitectura posterior a 2017	Intel Core i5/i7/i9 de 8.ª gen o Ryzen 5/6/7/9
Disco	8 GB libres (Studio) · 16 GB libres (Studio + emulador)	SSD con 32 GB o más
Resolución	1280 × 800	1920 × 1080
Tarjeta gráfica	Ninguna (solo Studio) · 4 GB de VRAM (con emulador)	8 GB de VRAM

Cuadro 7: Requisitos de Android Studio (Windows)

En macOS los mínimos son macOS 12, 8 GB de RAM y un Apple M1 o un Intel de 6.ª generación en adelante; en Linux, cualquier distribución de 64 bits y **glibc 2.31 o superior**. Tres advertencias: **Windows con procesador ARM no está soportado**, los **Intel Core N-Series y U-Series no se recomiendan**, y cada emulador adicional consume unos **4 GB de RAM** y **6 GB de disco**.

Paso 1 — Descargar el instalador.

Se descarga desde `https://developer.android.com/studio`. La versión estable es **Quail 4 | 2026.1.4 Patch 1**, y el instalador ocupa unos **1,5 GB**:

- **Windows**: `android-studio-quail4-patch1-windows.exe`.
- **macOS**: `...-mac.dmg` o `...-mac_arm.dmg`.
- **Linux**: `...-linux.tar.gz`.

- **ChromeOS**: `...-cros.deb`.

> **[CAPTURA PENDIENTE 3.1] — Figura 3.1: página de descarga.** Haz una captura de `developer.android.com/studio` en la que se vea la versión disponible y la lista de sistemas operativos. Si puedes, que se vea el botón de descarga y la versión al lado.

Paso 2 — Ejecutar el instalador.

- **Windows**: doble clic en el `.exe` y seguir el asistente.
- **macOS**: abrir el `.dmg` y **arrastrar el icono a Aplicaciones**.
- **Linux**: descomprimir en `/opt/` y lanzar `android-studio/bin/studio`.

Paso 3 — El asistente de configuración inicial (Setup Wizard).

El asistente ofrece dos modos de instalación:

- **Standard (estándar)**: instala automáticamente el conjunto recomendado.
- **Custom (personalizado)**: permite elegir componente a componente.

> **¿Por qué importa instalar el SDK?** Porque una cosa es el IDE (la interfaz) y otra es el SDK (las bibliotecas y herramientas de compilación). El IDE sin SDK no puede compilar nada. Por eso el asistente descarga todo de golpe, aunque tú no lo veas.

> **[CAPTURA PENDIENTE 3.2] — Figura 3.2: asistente de configuración.** Captura del *Setup Wizard*. Lo ideal es la pantalla de elección entre **Standard** y **Custom** o la de **resumen de componentes**.

> **[CAPTURA PENDIENTE 3.3] — Figura 3.3: pantalla principal de Android Studio.** Captura de la pantalla de bienvenida con un proyecto abierto, o de un proyecto vacío. Que se vea el árbol de proyectos a la izquierda.

3.2 Creación de un proyecto de prueba

Paso 1 — Abrir el asistente de nuevo proyecto. Botón **New Project** en la pantalla de bienvenida, o **File >New >New Project**.

Paso 2 — Elegir la plantilla. En el panel de plantillas, dentro de **Phone and Tablet**, hay que seleccionar **`Empty Views Activity`**.

> **Aviso importante.** En Android Studio conviven dos plantillas parecidas:
>
> - **`Empty Views Activity`** → genera la interfaz con **vistas XML** en `res/layout/`.
> - **`Empty Activity`** → genera la interfaz con **Jetpack Compose**, sin ningún XML.
>
> El enunciado pide explícitamente la de vistas XML, así que esa es la que hay que usar. Pero conviene que sepas que desde mayo de 2026 la posición oficial de Android Developers es que **el desarrollo de**

interfaces en Android es «Compose-first».

Paso 3 — Configurar el proyecto.

Campo	Valor a introducir	Por qué
Name	`PracticaEntornoMovil`	Nombre del proyecto y de la carpeta
Package name	`com.ejemplo.practicaentornomov`	Identificador único. No se puede cambiar después
Save location	Carpeta con espacio	El proyecto + emulador se llevan bastante
Language	**Kotlin**	El enunciado es explícito: debe ser Kotlin, no Java
Minimum API level	**API 24 (Android 7.0 Nougat)**	Valor que usa el codelab oficial de Google

Cuadro 8: Campos de configuración del nuevo proyecto

⚠ **Sobre la contradicción del enunciado.** En el punto 3.4 aparece «la selección de Java», mientras que en el 3.2 se dice explícitamente que el proyecto **debe ser en Kotlin, no en Java**. Es una errata. **Lo correcto es Kotlin**, y por eso el archivo se llama `MainActivity.kt` y no `MainActivity.java`.

📷 **[CAPTURA PENDIENTE 3.4] — Figura 3.4: ventana de configuración del proyecto nuevo.** Captura a pantalla completa de la ventana de **New Project** con los campos rellenos. Añade una imagen ampliada (recorte) de la zona del desplegable de **Language** con **Kotlin** seleccionado.

Paso 4 — Esperar a la sincronización.

Al pulsar **Finish**, Android Studio crea el proyecto y lanza una sincronización de Gradle en segundo plano. En la parte inferior verás la barra de progreso.

📷 **[CAPTURA PENDIENTE 3.5] — Figura 3.5: sincronización de Gradle.** Captura de la barra inferior de Android Studio con la tarea de Gradle en curso (o ya completada, sin errores en rojo).

3.3 Comprobación del funcionamiento

Paso 1 — Localizar la estructura del proyecto.

```
PracticaEntornoMovil/
 app/
    build.gradle.kts
    proguard-rules.pro
    src/main/
        AndroidManifest.xml
        java/com/ejemplo/practicaentornomovil/
           MainActivity.kt            ← ACTIVIDAD PRINCIPAL
        res/
            layout/
               activity_main.xml      ← DISEÑO DE LA INTERFAZ
```

```
            mipmap-anydpi-v26/
            values/
            xml/
 build.gradle.kts
 gradle.properties
 settings.gradle.kts
```

Paso 2 — Identificar la actividad principal y el diseño.

Archivo	Ruta	Qué contiene
Actividad principal	`app/src/main/java/.../MainActivity.kt`	La clase `MainActivity` y su `onCreate`, que llama a `setContentView`
Diseño	`app/src/main/res/layout/activity_`	El XML con la jerarquía de vistas

Cuadro 9: Archivos principales del proyecto

Paso 3 — Ejecutar en el emulador. Antes de nada, confirmar que la **virtualización por hardware está activada en la BIOS**:

```
emulator -accel-check
```

Requisitos oficiales del emulador: **16 GB de RAM** y **16 GB de espacio en disco**. Para crear el dispositivo virtual:

1. Abrir **Device Manager**.
2. Pulsar **+** y elegir **Create Virtual Device**.
3. Elegir un perfil de **Pixel**.
4. Elegir la **imagen de sistema**.
5. Revisar la configuración y pulsar **Finish**.

Para ejecutar: botón **Run** (o **Shift + F10**).

[CAPTURA PENDIENTE 3.6] — Figura 3.6: estructura del proyecto en Android Studio. Captura del panel izquierdo con la vista **Project** desplegada. Añade una **imagen ampliada** de la zona del árbol, porque los nombres de archivo tienen que leerse con claridad.

[CAPTURA PENDIENTE 3.7] — Figura 3.7: contenido de `MainActivity.kt` y de `activity_main.xml`. Dos capturas en las que se vea el contenido de cada archivo. En `MainActivity.kt` se ve que la extensión es `.kt` (prueba de que es Kotlin).

[CAPTURA PENDIENTE 3.8] — Figura 3.8: emulador con la aplicación ejecutada. Captura del emulador con la aplicación abierta. Si no te funciona el emulador, haz en su lugar una captura del **Device Manager** y explica el problema en el punto 3.4.

3.4 Problemas frecuentes y cómo solucionarlos

Cuadro 10: Problemas frecuentes durante la instalación y su solución

Problema	Causa	Solución
El emulador **no arranca**	No hay virtualización por hardware activada en la BIOS	Activar Intel VT-x o AMD-V en la BIOS y reiniciar. Comprobar con `emulator -accel-check`
El emulador va **muy lento** o se cuelga	Funciona sin aceleración de VM	Con virtualización activada se multiplica la velocidad. Alternativa: `emulator -gpu software`, dispositivo físico por USB, o Android Device Streaming
Error «Connection to the Internet denied»	Bloqueo de red al descargar dependencias de Gradle	Añadir a `gradle.properties`: `org.gradle.jvmargs=-Djava.net.preferIPv4Stack=true` y reiniciar
No arranca por falta de espacio	El emulador no arranca con menos de 5 GB libres	Liberar espacio en disco
Las apps de Google **fallan en API reciente**	Imagen de sistema sin el parche de certificación	Actualizar la imagen del sistema a la revisión 5 o posterior de la API 37
El **antivirus ralentiza** el emulador	El antivirus intercepta lecturas y escrituras	Añadir el emulador a la lista de aplicaciones de confianza
La **multitáctil no funciona**	La ventana de herramientas no soporta multitáctil	Ejecutar el emulador en su propia ventana separada
El emulador **no arranca anidado** en otra VM	Los emuladores con aceleración no pueden correr dentro de VirtualBox, VMware o Docker	Ejecutarlo en la máquina real, no en una VM
El proyecto aparece con **errores en rojo**	Las dependencias aún no se han descargado	Esperar a la sincronización. Si persiste: **File >Invalidate Caches and Restart**
La resolución DPI se detecta **mal**	Detección automática de DPI fallida	Añadir `hidpi=true` o `hidpi=false` en `gradle.properties`

Fuentes del apartado 3: Android Studio · Install Android Studio · Create a new project · Project templates · uses-sdk element · Run the Android Emulator · Hardware acceleration · Create and manage AVDs · Emulator troubleshooting · Android Studio troubleshooting

3.5 Lista de evidencias pendientes de capturar

#	Figura	Qué tiene que verse	Prioridad
1	3.1	Página de descarga de Android Studio con la versión disponible	Alta
2	3.2	Asistente de configuración (Standard/Custom o resumen)	Media
3	3.3	Pantalla principal de Android Studio	Alta
4	3.4	Ventana de New Project con **Language = Kotlin** (y recorte)	**Muy alta**
5	3.5	Sincronización de Gradle terminada sin errores	Media
6	3.6	Árbol del proyecto con `MainActivity.kt` y `activity_main.xml`	**Muy alta**
7	3.7	Contenido de `MainActivity.kt` y de `activity_main.xml`	**Muy alta**
8	3.8	Emulador con la app ejecutada (o Device Manager + explicación)	Alta

Cuadro 11: Lista consolidada de capturas pendientes

Recuerda. Las capturas a pantalla completa cuando hagan falta, resolución suficiente para que se lea el texto, **una imagen ampliada de la zona relevante** cuando algo no se aprecie, y **un pie de imagen bajo cada captura** que explique qué se está mostrando.

4 Identificación de configuraciones de dispositivos móviles

En el Apartado 1 hablé de las limitaciones en abstracto y en el Apartado 2 de las herramientas. Este apartado es el puente entre las dos cosas: **qué datos concretos hay que mirar de un dispositivo para saber si una aplicación va a funcionar bien en él**. Los cinco criterios son: **resolución y densidad**, **sistema operativo y versión**, **almacenamiento interno**, **memoria RAM y potencia de procesamiento** y **conectividad**.

> **Nota de método.** En los cinco criterios que siguen separo lo que la documentación oficial dice literalmente de lo que es criterio propio mío. Cuando es lo segundo, lo digo.

4.1 Resolución de pantalla y densidad

¿En qué consiste? Una pantalla no se mide solo en píxeles. Se mide en **píxeles físicos**, en **densidad** y en **dp**. La cuenta oficial:

```
px = dp × (dpi / 160)
```

La documentación lo dice así: «One dp is a virtual pixel unit that's roughly equal to one pixel on a medium-density screen (160 dpi, or the 'baseline' density)».

Grupo	Nombre	dpi aprox.	Factor	Icono 48dp
ldpi	Baja densidad	∼120	0,75×	36×36 px
mdpi	Densidad media (**ref.**)	∼160	1,0×	48×48 px
hdpi	Alta densidad	∼240	1,5×	72×72 px
xhdpi	Extra alta	∼320	2,0×	96×96 px
xxhdpi	Extra extra alta	∼480	3,0×	144×144 px
xxxhdpi	Extra extra extra alta	∼640	4,0×	192×192 px

Cuadro 12: Grupos de densidad de Android

¿Por qué afecta a una aplicación? Porque **el mismo número de píxeles no significa lo mismo en dos teléfonos distintos**.

¿Cómo lo clasifico? Android agrupa las pantallas por **tamaño en dp**:

Desde **Android 17 (API 37)** ya no se pueden declarar bloqueos de orientación o de redimensionado en pantallas grandes (sw $\geq$ 600dp).

Conclusión del criterio. La resolución por sí sola sirve poco; lo que hay que mirar es la **densidad** y el **tamaño en dp**.

> Support different screen densities · Configuration.screenLayout · Screen sizes · Android 17 is here

	Móvil antiguo gama baja	Móvil gama alta actual
Resolución típica	720 × 1440 px	1344 × 2992 px
Densidad típica	`xhdpi` (∼320 dpi) → 2,0×	`xxxhdpi` (∼640 dpi) → 4,0×
`DisplayMetrics.density`	2,0	4,0
Un icono de 48dp se dibuja en	96 × 96 px	192 × 192 px
Consecuencia	Se ve correcto	Si falta la versión alta, **escala y se ve borroso**

Cuadro 13: Comparación de un mismo icono en dos densidades

Constante	Etiqueta	Tamaño mínimo
`SCREENLAYOUT_SIZE_SMALL`	`small`	320 × 426 dp
`SCREENLAYOUT_SIZE_NORMAL`	`normal`	320 × 470 dp
`SCREENLAYOUT_SIZE_LARGE`	`large`	480 × 640 dp
`SCREENLAYOUT_SIZE_XLARGE`	`xlarge`	720 × 960 dp

Cuadro 14: Tamaños de pantalla en Android

4.2 Sistema operativo y versión

¿En qué consiste? El sistema operativo es lo primero que hay que mirar. En Android hay dos números distintos: **el nombre de la versión** y **el nivel de API**. La documentación avisa: «The API level is always a single integer. You can't derive the API level from its associated Android version number».

Versión de Android	Nivel de API
Android 17	**37**
Android 16	**36**
Android 15	**35**
Android 14	**34**
Android 13	**33**
Android 12L / 12	32 / **31**
Android 11	**30**
Android 10	**29**
Android 9	**28**
Android 8.1 / 8.0	27 / 26
Android 7.1 / 7.0	25 / 24

Cuadro 15: Correspondencia versión de Android y nivel de API

¿Por qué afecta a una aplicación? Tres razones:

1. **Si no subes el `targetSdkVersion`, Google Play te deja fuera.** Desde el **31 de agosto de 2026**, las aplicaciones nuevas y actualizaciones tienen que apuntar a Android 16 (API 36) o superior.

2. **Las apps antiguas se vuelven invisibles.** Para seguir siendo visibles a usuarios nuevos: mínimo **Android 15 (API 35)**.

3. **El sistema cambia de comportamiento sin avisar.**

Versión	Permiso necesario para fotos/vídeos
Android 10 (API 29)	`READ_EXTERNAL_STORAGE` / `WRITE_EXTERNAL_STORAGE`
Android 11+ (API 30+)	`READ_EXTERNAL_STORAGE` (vía `MediaStore`)
API $\leq$ 28	Acceso directo a todos los archivos

Cuadro 16: Permisos cambiantes según versión

¿Cómo lo clasifico?

Tramo	Versión aproximada	Qué esperar
Antiguo / mantenimiento	Android 9-11 (API 28-30)	Mucha gente lo usa. Evitar APIs modernas sin `if` de compatibilidad
Estable / mayoritario	Android 12-15 (API 31-35)	La base razonable de usuarios hoy
Reciente	Android 16-17 (API 36-37)	Novedades, límites de memoria por RAM
Futuro / riesgo	API $\geq$ 38 (sin publicar)	Leer *behavior changes* en cuanto salga

Cuadro 17: Tramos de antigüedad del SO

En el proyecto de prueba: **`minSdk` = 24**, **`targetSdk` = el del SDK instalado** y **`compileSdk`** igual al `targetSdk`.

> Android API levels · Android 17 release notes · Behavior changes · uses-sdk · Target API requirements · Apple Security Releases

4.3 Almacenamiento interno

¿En qué consiste? El almacenamiento interno es el espacio del teléfono donde van las apps y sus datos. Hay que distinguir **la capacidad del dispositivo** y **el espacio disponible para una aplicación concreta**. Android no publica tabla oficial de «gamas de almacenamiento», así que las franjas son **convención de mercado**.

Gama	Total	Útil	Perfil típico
Baja	16-32 GB	8-10 GB	Móvil antiguo o de entrada
Media	64-128 GB	40-70 GB	La gama mayoritaria
Alta	256 GB - 1 TB	200 GB+	Gama alta

Cuadro 18: Gamas de almacenamiento (convención)

La documentación oficial avisa: «Internal storage has limited space for app-specific data. Use other types of storage if you need to save a substantial amount of data».

Contenido	Acceso	Permisos	¿Otras apps?	¿Se borra?
Archivos propios	`getFilesDir()`, `getCacheDir()`	Nunca	No	**Sí**
Multimedia	API `MediaStore`	`READ_EXTERNAL_STORAGI`	Sí, con permiso	**No**
Documentos	Storage Access Framework	Ninguno	Sí, vía selector	No
Preferencias	Jetpack Preferences	Ninguno	No	Sí
Bases de datos	Room	Ninguno	No	Sí

Cuadro 19: Tipos de almacenamiento y comportamiento

¿Cómo lo clasifico? Por el tamaño del APK/AAB:

Gama	Tamaño aceptable	Estrategia
Baja	Máx. ∼100 MB	Todo lo pesado descargado bajo demanda
Media	Hasta ∼500 MB	Descarga bajo demanda + AAB
Alta	Limitado por red, no disco	Más contenido local permitido

Cuadro 20: Estrategia de tamaño por gama

Storage overview · APK Analyzer · App Bundles · Install Android Studio · Emulator troubleshooting

4.4 Memoria RAM y potencia de procesamiento

¿En qué consiste? La RAM es el espacio donde se ejecutan los procesos. Android tiene una **tabla oficial de umbrales de «mal comportamiento» según la gama de RAM**.

RAM física	Primer plano	Servicios visibles	Segundo plano
4 GB	**2,00 GB**	1,00 GB	1,00 GB
6 GB	**2,25 GB**	1,25 GB	1,25 GB
8 GB	**2,25 GB**	1,50 GB	1,50 GB
12 GB	**3,25 GB**	1,75 GB	1,75 GB
16 GB	**4,25 GB**	2,00 GB	2,00 GB

Cuadro 21: Umbrales de *Android vitals* para apps

Y un dato importantísimo: **los límites de memoria para imágenes (*bitmaps*) no varían con la gama de RAM**. Son 200 MB en servicios y segundo plano, y 400 MB en caché, **siempre**.

¿Por qué afecta a una aplicación? Android da las herramientas para preguntarle al dispositivo cómo es:

¿Cómo lo clasifico?

API	Qué te dice
`ActivityManager.getMemoryClass()`	«Return the approximate per-application memory class of the current device»
`ActivityManager.getLargeMemoryClass`	El límite si declaras `android:largeHeap="true"`
`ActivityManager.isLowRamDevice()`	Si el dispositivo es de poca memoria
`ActivityManager.getMemoryInfo(outIn`	Estado general de la memoria del sistema

Cuadro 22: APIs de Android para consultar la memoria

Tramo	RAM física		Qué hacer
Crítico	$\leq$ 2 GB	`true`	Cachés mínimos, sin `largeHeap`, una imagen cada vez
Bajo	3-4 GB	`true`	Imágenes reducidas, paginación, 2 hilos máx.
Medio	6-8 GB	`false`	Comportamiento estándar
Alto	12 GB+	`false`	Todo con margen para más trabajo concurrente

Cuadro 23: Clasificación por gama de RAM

> Android vitals · ActivityManager · Behavior changes Android 17 · Guide to Android app architecture · Install Android Studio

4.5 Conectividad

¿En qué consiste? La conectividad es, con diferencia, **el recurso más variable de los cinco**. Hay cuatro familias: **datos móviles**, **Wi-Fi**, **Bluetooth** (dos variantes) y **NFC**. Y un quinto factor transversal: **el hecho de que la red sea «medida» (*metered*)**.

¿Por qué afecta a una aplicación?

Red de datos. Android soporta 5G desde Android 10:

Caso de uso oficial	Qué implica para la app
Indoor $\rightarrow$ *Outdoor*	Videollamada en movimiento, descargas bajo demanda
Photo AR	Pasar de foto a vídeo o RA sin salir de la app
Prefetch	«Use increased bandwidth to prefetch content — delight rather than buffer»
Niche $\rightarrow$ *mainstream*	El streaming deja de ser el caso raro

Cuadro 24: Casos de uso oficiales del 5G

Los tres «trabajos» oficiales: **comprobar si la red es medida** (`isActiveNetworkMetered()`), **detectar 5G** y **estimar el ancho de banda**.

Bluetooth. Dos cosas: **clásico** (streaming, archivos) y **Bluetooth Low Energy (BLE)** (sensores, beacons, wearables). Modelo de emparejamiento: descubrible $\rightarrow$ descubrimiento de servicios $\rightarrow$ *bonding*.

Bluetooth LE Audio (LEA) está en Android 13 (API 33): códec **LC3**, audio compartido, **Auracast**, **Hearing Aid Profile** y **EATT**.

Wi-Fi.

Estándar	Norma IEEE	Desde	API
Wi-Fi 6	802.11ax	Android 9+	`ScanResults.WIFI_STANDARD_11AX`
Wi-Fi 6E	802.11ax (6 GHz)	Compatibles	`WifiManager.is6GHzBandSupported()`
Wi-Fi 7	802.11be	Android 13+	`ScanResults.WIFI_STANDARD_11BE`

Cuadro 25: Estándares Wi-Fi en Android

NFC. «A set of short-range wireless technologies, typically requiring a distance of 4 cm or less». Tres modos: lector/escritor, emulación de tarjeta e interoperabilidad entre dispositivos.

¿Cómo lo clasifico? Como **tabla de decisiones**:

Situación	Decisión de diseño
Solo datos móviles medidos	No descargar nada grande sin preguntar. `isActiveNetworkMetered()`
Wi-Fi sin coste	Permitir descargas pesadas y *prefetch*
Ancho de banda bajo	Bajar la calidad del vídeo automáticamente
Bluetooth clásico	Reproducción de audio de alta calidad
Solo BLE	Sensores y beacons; nada de audio alta fidelidad
LE Audio (API 33+)	Ofrecer Auracast y perfil de audífonos
NFC disponible	Pagos contactless y lectura de etiquetas
Sin conexión	Diseño *offline-first*
Android 17+	Migrar de `usesCleartextTraffic` a config. de seguridad de red

Cuadro 26: Tabla de decisiones según la conectividad

5G · Bluetooth · Bluetooth LE Audio · NFC · Wi-Fi · Wi-Fi 7

4.6 Tabla resumen de los cinco criterios

Cuadro 27: Resumen de los cinco criterios

#	Criterio	Dato a mirar	Umbral / referencia	Si falla, síntoma
1	**Resolución y densidad**	Densidad, tamaño en `dp`	`small` 320×426, `normal` 320×470, `large` 480×640, `xlarge` 720×960	Imágenes borrosas o desproporcionadas; texto ilegible
2	**SO y versión**	API del dispositivo, `minSdk`, `targetSdk`	Play exige `targetSdk` ≥ 36 desde 31/08/2026; ≥ 35 visible	La app no aparece en Play o no se ve en teléfonos nuevos
3	**Almacenamiento**	Espacio libre, tamaño AAB	Emulador: mín. 5 GB libres. Studio: 8 GB (16 GB con emulador)	Fallo al instalar; pérdida de datos al desinstalar

#	Criterio	Dato a mirar	Umbral / referencia	Si falla, síntoma
4	**RAM y CPU**	Gama de RAM, `getMemoryClass()`, `isLowRamDevice()`	Vitals (primer plano): 2,00 GB en 4 GB → 4,25 GB en 16 GB. Imágenes: 200/400 MB fijos	Cierre automático, interfaz a saltos, `OutOfMemoryError`
5	**Conectividad**	4G/5G, Wi-Fi, BT, NFC, si es medida	Wi-Fi 7 desde Android 13; LE Audio desde Android 13; NFC $\leq$ 4 cm	Buffers, descargas que consumen datos, funciones inservibles

Una sola regla. Todos estos criterios se pueden consultar en tiempo de ejecución. `DisplayMetrics` para la pantalla, `Build.VERSION.SDK_INT` para la versión, `StatFs` para el espacio, `ActivityManager` para la memoria, `ConnectivityManager` para la red. Una aplicación bien hecha **pregunta al dispositivo**, no asume.

Fuentes del apartado 4: Screen densities · Screen sizes · Configuration.screenLayout · Android 17 is here · API levels · Behavior changes · uses-sdk · Target API requirements · Storage overview · APK Analyzer · App Bundles · Android vitals · ActivityManager · 5G · Bluetooth · Bluetooth LE Audio · NFC · Wi-Fi · Wi-Fi 7

4.7 Conclusión del apartado 4

Cuatro ideas:

Primera: el número grande no sirve, la medida correcta sí.

Segunda: los límites de Android no son los mismos para todos, pero algunos sí. Los límites de memoria *totales* suben con la gama de RAM (de 2,00 GB a 4,25 GB), pero **los límites de memoria para imágenes son idénticos en todos los dispositivos: 200 MB y 400 MB**. Consecuencia práctica: **optimizar el consumo de *bitmaps* es la optimización que mejora a todos los usuarios a la vez**.

Tercera: hay fechas, y las fechas mandan. El 31 de agosto de 2026 Google Play exige `targetSdk` $\geq$ 36.

Cuarta: el mejor diseño es el que pregunta. Ninguno de estos cinco criterios obliga a hacer una apuesta: todos se pueden consultar en tiempo de ejecución.

Con esto ya sé **qué mirar del dispositivo**. Falta la otra mitad: **para quién se programa**.

5 Establecimiento de perfiles de usuario

Este es el apartado donde todo lo anterior se cruza. Hasta aquí he hablado de dispositivos: cuántos gigabytes, qué versión, cuántos píxeles. Pero un móvil no lo usa una persona, lo usa **alguien**, y ese alguien tiene necesidades que no aparecen en ninguna ficha técnica.

¿En qué consiste un perfil de usuario? Un perfil de usuario es, simplemente, **una descripción de quién va a usar la aplicación y en qué condiciones**. Cada perfil obliga a tomar decisiones de diseño concretas.

Los seis perfiles que he definido se combinan con los cinco criterios del Apartado 4. Un perfil no dice «persona de 67 años» y punto, sino **persona de 67 años con un móvil de gama media de hace tres años, con conectividad intermitente y de coste alto**.

5.1 Tabla de perfiles

Cuadro 28: Tabla de perfiles de usuario

#	Perfil	Contexto	Dispositivo esperado	Accesibilidad	Decisiones
P1	**Usuario general**	Uso diario de redes, banca, compras, mensajería	Gama media o alta. Android 12+. 64 GB+, 6 GB RAM	Ninguna especial	Diseño base. Descargas bajo demanda en Wi-Fi, caché razonable, `targetSdk` al día
P2	**Usuario gama baja / antiguo**	Uso básico: WhatsApp, cámara, alguna app sencilla	Android 9-11. 32 GB, 3 GB RAM, 4G intermitente, `hdpi`	Baja agudeza visual por edad o pantalla pequeña	Imágenes comprimidas; sin `largeHeap`; paginación; mín. 2 hilos; textos en `sp`
P3	**Persona mayor**	Llamadas, turnos médicos, calculadora, banco	Gama media, a veces baja. Android 11+	**Tamaño de fuente grande, contraste alto, botón de emergencia**	Objetivos táctiles 48×48dp; tipografía grande; contraste $\geq$ 4,5:1; sin gestos complejos
P4	**Usuario con discapacidad visual**	Cualquiera, con lector de pantalla activo	Gama alta normalmente (no necesariamente)	**TalkBack, VoiceOver, Braille, Switch Access**	Etiquetas en todos los elementos; orden lógico; no depender solo del color; audio y vídeo descritos
P5	**Usuario con discapacidad auditiva**	Llamadas, vídeo, videollamadas	Cualquiera	**Subtítulos, Auracast, HAP, vibración**	Subtítulos/transcripción; compatibilidad LEA desde Android 13; avisar por vibración
P6	**Usuario en movilidad**	Tren, ascensor, sótano, viaje	Cualquiera	Compartir ubicación	Diseño *offline-first*; caché local; reintentos; `isActiveNetworkMetered()`

Nota de coherencia con el Apartado 4. Los perfiles P2 y P6 son los que más se relacionan con los cinco criterios técnicos. Un móvil de gama baja (criterio 4.4) es, casi siempre, un móvil con poco almacenamiento (4.3), versión antigua (4.2) y pantalla `hdpi` (4.1). Los perfiles no son categorías de personas, sino **combinaciones de persona + dispositivo + contexto**.

5.2 Perfil de accesibilidad y diseño: los números concretos

De los seis perfiles, el **P4 (baja visión o ceguera)** merece un apartado propio. Los cinco parámetros que hay que cumplir:

1. Tamaño del objetivo táctil: 48dp × 48dp mínimo

«For touch interfaces, we recommend that each interactive UI element have a focusable area, or touch target size, of at least 48dp × 48dp. Larger is even better.»

Un matiz importante: **el área táctil no es lo que se ve**. Un icono puede aparecer como 24×24dp pero el *padding* alrededor completa el objetivo de 48×48dp. Se recomienda además **8dp o más de separación** entre objetivos.

Guía	Tamaño mínimo del objetivo
Android / Material Design	**48 × 48 dp**
WCAG 2.2 (W3C), criterio 2.5.8	24 × 24 píxeles CSS
iOS (HIG)	44 × 44 dp

Cuadro 29: Tamaños mínimos de objetivo entre guías

2. Contraste de color: 4,5:1 en texto pequeño y 3:1 en texto grande

Tipo de texto	Contraste mínimo
Texto **menor de 18sp**, o negrita **menor de 14sp**	$\geq$ **4,5 : 1**
Texto grande ($\geq$ 18sp, o $\geq$ 14sp en negrita) y gráficos	$\geq$ **3 : 1**
Elementos no textuales (marco de botón) frente al fondo	$\geq$ **3 : 1**

Cuadro 30: Requisitos de contraste (WCAG 2.2 / Material)

3. El texto tiene que poder crecer hasta el 200 %

WCAG 2.2, criterio **1.4.4 Resize Text (AA)**: el texto se puede ampliar «without assistive technology up to 200 percent without loss of content or functionality». En Android es el ajuste **Tamaño de fuente**. Para que crezca sin romper el diseño hay que escribirlo en **sp**, nunca en dp ni en px.

4. Etiquetas en todo, y TalkBack tiene que poder describirlo

5. Detalle para P5 (discapacidad auditiva)

Bluetooth LE Audio (LEA) está integrado en Android desde la versión 13:

Principios oficiales	Qué significa en código
Etiquetar los elementos	«Screen readers such as TalkBack can announce these labels to users». `semantics` en Compose; `contentDescription` en Views
Añadir acciones de accesibilidad	«By adding accessibility actions, you can enable users of accessibility services to complete critical user flows»
Usar las funciones ya incorporadas	«Compose offers many accessibility behaviors by default»
Usar señales además del color	«use patterns and position, along with color, to express these differences»
Hacer el contenido multimedia accesible	Describir audio y vídeo para que los usuarios no dependan solo de señales visuales o sonoras

Cuadro 31: Principios oficiales de accesibilidad

Aspecto	Comportamiento oficial de TalkBack
Qué es	Lector de pantalla del sistema: «announces content through a synthesized voice and performs actions on an app in response to user gestures»
Cómo se activa	*Ajustes >Accesibilidad >TalkBack*; o pulsando los dos botones de volumen
Gestos	Toque = seleccionar; doble toque = abrir; deslizar derecha = siguiente; dos dedos arriba/abajo = desplazarse
Braille	Teclado Braille de 6 puntos incluido en TalkBack
Servicio complementario	**Switch Access**, para «people who have motor disabilities… controlling the device using only one or two buttons»

Cuadro 32: Comportamiento de TalkBack

Función	Qué aporta
Códec **LC3**	Más calidad con menos batería que el SBC
Auracast / Broadcast Audio	Difundir audio a audífonos, personas cercanas y terminales públicos
Hearing Aid Profile (HAP)	«Greater accessibility and usage than previous ASHA protocols»: audífonos usables en llamadas y VoIP
EATT	Permite enviar varios comandos a la vez a los audífonos
Comprobación en código	`isLeAudioSupported()` y `isLeAudioBroadcastSourceSupported()`

Cuadro 33: Bluetooth LE Audio y accesibilidad auditiva

Fuentes del apartado 5.2: Make your app more accessible · Principles for improving app accessibility · Accessibility in Jetpack Compose · Accessibility on your Android device · Change font size and display size · Use TalkBack · Use braille with TalkBack · Bluetooth LE Audio · WCAG 2.2 (W3C) · Material Design 3: color contrast · Material Design 3: structure

5.3 Reflexión final: por qué perfiles y dispositivo van juntos

La idea: el dispositivo y la persona no se analizan por separado.

	Gama baja (P2)	Gama alta (P1)
Sin necesidades especiales (P1/P6)	El caso mayoritario, y el que casi siempre se olvida en las pruebas	El caso que todo el mundo prueba
Con necesidades de accesibilidad (P3/P4/P5)	**El más difícil de todos**: dispositivo antiguo + TalkBack + fuente al 200 %	Necesidades de accesibilidad sobre un buen dispositivo: se resuelve bien casi siempre

Cuadro 34: Matriz de combinaciones dispositivo × perfil

La casilla de abajo a la izquierda es la que resume el problema entero. Un usuario con baja visión en un móvil antiguo:

- su móvil tiene **3 GB de RAM**: la app tiene que funcionar con la mitad de caché;
- su pantalla es `hdpi`: los recursos tienen que estar bien optimizados;
- lleva **Android 10**: los permisos son los antiguos;
- tiene el **texto al 180 %**: todo debe estar en `sp` y con layouts flexibles;
- y usa **TalkBack**: cada elemento necesita etiqueta y el orden de lectura importa.

Segunda idea: la accesibilidad no es un extra, es la versión más restrictiva del diseño responsive.

Tercera idea: casi todo se puede preguntar en tiempo de ejecución. `DisplayMetrics`, `Build.VERSION.SDK_INT`, `StatFs`, `ActivityManager`, `ConnectivityManager`, `isLeAudioSupported()`, `isLowRamDevice()`, `isActiveNetworkMet`
No hace falta adivinar: se pregunta.

Conclusiones finales

1. Las limitaciones del móvil se pueden enunciar en una frase y resolver en veinte decisiones. La frase es: *recursos finitos y compartidos, energía finita y conexión intermitente*. Las veinte decisiones son: tamaños en `dp` y texto en `sp`, recursos por densidad, gráficos vectoriales, `minSdk` conservador y `targetSdk` actualizado, `App Bundle` en vez de un APK gigante, contenido pesado descargado bajo demanda, caché de *bitmaps* acotada, `isLowRamDevice()` como interruptor, paginación en lugar de listas interminables, *offline-first*, `isActiveNetworkMetered()` como puerta de las descargas, objetivos de 48dp, contraste de 4,5:1, texto ampliable al 200 %, etiquetas para TalkBack, alternativas a los gestos, subtítulos, y así hasta veinte. **Ninguna de las limitaciones es un motivo para no hacer una app; son una lista de requisitos.**

2. Elegir tecnología es elegir qué problemas quieres tener. Entre Java, Kotlin, Flutter, React Native, .NET MAUI y Swift, la decisión no es «cuál es mejor» (no hay respuesta única) sino «qué acepto». Ninguna opción es «la correcta»: cada una mueve el coste a un sitio distinto.

3. El entorno de desarrollo también es una decisión de diseño. Instalar Android Studio, fijar `minSdk` = 24, elegir Kotlin, comprobar que el emulador arranca con aceleración por hardware y capturar la evidencia: todo eso no es «preparar la máquina», es decidir contra qué se va a desarrollar. En el enunciado aparece una contradicción (Java vs Kotlin); **me quedo con Kotlin**, que es el lenguaje oficial recomendado por Google desde 2019.

4. Lo que no se mide en el dispositivo se mide en la persona. Es la conclusión del Apartado 5 y la que más me ha cambiado el planteamiento. Dedicar el 20 % del trabajo a densidad, API, RAM y conectividad, y el otro 80 % a pensar para quién se programa (edad, baja visión, motora, conectividad intermitente) es lo que separa una app funcional de una app buena.

5. Lo más útil que he aprendido: preguntar en vez de suponer. Cada apartado termina con la misma idea desde un ángulo distinto. *Android vitals* publica umbrales de memoria **en función de la RAM del dispositivo**, para que no tengas que adivinar cuánto puede usar tu app. El sistema expone `isLowRamDevice()`, `getMemoryClass()`, `isActiveNetworkMetered()`, `isLeAudioSupported()` para lo mismo. TalkBack existe para probar tu app sin vista. El **Accessibility Scanner** existe para encontrar contraste bajo y objetivos táctiles demasiado pequeños automáticamente. Y el emulador existe para crear un dispositivo de gama baja y 3 GB de RAM **en tu ordenador**. **Ninguna de estas herramientas sustituye a pensar; todas evitan tener que adivinar**, y ese es el cambio de mentalidad que me llevo de esta actividad.

Referencias bibliográficas

Todas las fuentes que aparecen en este trabajo son **documentación oficial del fabricante o de la plataforma** (Android Developers, Google Play Console, Apple, Material Design, W3C, Bluetooth SIG).

> ⚠ **Nota importante sobre las fechas.** Todas las páginas se han consultado el **1 de octubre de 2026**. Los datos de versiones (niveles de API, políticas de Google Play, requisitos de Android Studio, cifras de almacenamiento) **cambian con frecuencia**: antes de entregar el trabajo, vuelve a abrir al menos estas cuatro páginas:
>
> - *Android API levels*
> - *Target API level requirements* (Google Play)
> - *Android Studio* (versión y requisitos)
> - *Android vitals* (umbrales de memoria)

Fuentes de Android Developers (documentación técnica)

Android Developers. (2026). *Android vitals*. https://developer.android.com/topic/performance/vitals

Android Developers. (2026). *Support different screen densities*. https://developer.android.com/training/multiscreen/screendensities

Android Developers. (2026). *Screen sizes*. https://developer.android.com/training/multiscreen/screensizes

Android Developers. (2026). *Configuration.screenLayout*. https://developer.android.com/reference/android/content/res/Configuration

Android Developers. (2026). *Android API levels*. https://developer.android.com/tools/releases/platforms

Android Developers. (2026). *uses-sdk element*. https://developer.android.com/guide/topics/manifest/uses-sdk-elemen

Android Developers. (2026). *Android 17 is here*. https://developer.android.com/blog/posts/android-17-is-here

Android Developers. (2026). *Behavior changes: all apps*. https://developer.android.com/about/versions/17/behavior-changes-all

Android Developers. (2026). *Android 17 release notes*. https://developer.android.com/about/versions/17/release-notes

Android Developers. (2026). *Storage overview*. https://developer.android.com/training/data-storage

Android Developers. (2026). *APK Analyzer*. https://developer.android.com/studio/debug/apk-analyzer

Android Developers. (2026). *Android App Bundles*. https://developer.android.com/guide/app-bundle/dynamic-delivery

Android Developers. (2026). *ActivityManager*. https://developer.android.com/reference/android/app/ActivityManager

Android Developers. (2026). *Guide to Android app architecture*. https://developer.android.com/topic/libraries/architecture

Android Developers. (2026). *5G*. https://developer.android.com/develop/connectivity/5g

Android Developers. (2026). *Bluetooth*. https://developer.android.com/develop/connectivity/bluetooth

Android Developers. (2026). *Bluetooth LE Audio overview*. https://developer.android.com/develop/connectivity/bluetooth/ble-audio/overview

Android Developers. (2026). *Near Field Communication*. https://developer.android.com/develop/connectivity/nfc

Android Developers. (2026). *Wi-Fi*. https://developer.android.com/develop/connectivity/wifi
Android Studio Community. (2026). *Wi-Fi 7*. https://source.android.com/docs/core/connect/wifi-7

Fuentes de Android Studio (Apartado 3)

Android Developers. (2026). *Android Studio*. https://developer.android.com/studio
Android Developers. (2026). *Install Android Studio*. https://developer.android.com/studio/install
Android Developers. (2026). *Create a new Android Studio project*. https://developer.android.com/studio/projects/create-project
Android Developers. (2026). *Android Studio project templates*. https://developer.android.com/studio/projects/templates
Android Developers. (2026). *Run the Android Emulator*. https://developer.android.com/studio/run/emulator
Android Developers. (2026). *Configure hardware acceleration*. https://developer.android.com/studio/run/emulator-acceleration
Android Developers. (2026). *Create and manage Android virtual devices*. https://developer.android.com/studio/run/managing-avds
Android Developers. (2026). *Android Emulator troubleshooting*. https://developer.android.com/studio/run/emulator-troubleshooting
Android Developers. (2026). *Android Studio troubleshooting*. https://developer.android.com/studio/troubleshoot
Android Developers. (2026). *Android UI development is Compose-first* [blog, 19/05/2026]. https://developer.android.com/blog/posts/android-ui-development-is-compose-first

Fuentes de accesibilidad y diseño

Android Developers. (2026). *Make your app more accessible*. https://developer.android.com/guide/topics/ui/accessibility/apps
Android Developers. (2026). *Principles for improving app accessibility*. https://developer.android.com/guide/topics/ui/accessibility/principles
Android Developers. (2026). *Accessibility in Jetpack Compose*. https://developer.android.com/develop/ui/compose/accessibility
Android Developers. (2026). *TalkBack y accesibilidad en Android*. https://support.google.com/accessibility/android/answer/6006564
Android Developers. (2026). *Use TalkBack*. https://support.google.com/accessibility/android/answer/6007100
Android Developers. (2026). *Use braille with TalkBack*. https://support.google.com/accessibility/android/answer/16323943
Android Developers. (2026). *Change font size and display size*. https://support.google.com/accessibility/android/answer/11183305
Google. (2026). *Target API level requirements*. https://support.google.com/googleplay/android-developer/answer/11926878

Fuentes de estándares de diseño y accesibilidad

World Wide Web Consortium (W3C). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/

Material Design 3. (2026). *Color contrast*. https://m3.material.io/foundations/designing/color-contrast

Material Design 3. (2026). *Structure*. https://m3.material.io/foundations/designing/structure

Material Design 3. (2026). *Grids and spacing*. https://m3.material.io/foundations/layout/grids-spacing/density

Fuentes de fabricantes

Samsung Electronics. (2026). *Galaxy S26 Ultra — Especificaciones técnicas*. https://www.samsung.com/

Google. (2026). *Pixel 10 Pro XL — Especificaciones técnicas*. https://store.google.com/

OnePlus. (2026). *OnePlus 13 — Especificaciones*. https://www.oneplus.com/

Apple Inc. (2026). *iPhone 17 Pro Max — Especificaciones técnicas*. https://www.apple.com/iphone-17-pro-max/technical-specs/

Motorola. (2026). *moto g47 — Especificaciones técnicas*. https://www.motorola.com/

Apple Inc. (2026). *Apple Security Releases*. https://support.apple.com/en-us/148282

Fuentes de lenguajes y frameworks del Apartado 2

Kotlin Foundation. (2026). *Kotlin documentation*. https://kotlinlang.org/docs/home.html

Oracle. (2026). *Java Documentation*. https://docs.oracle.com/en/java/

Flutter. (2026). *Flutter documentation*. https://docs.flutter.dev/

Meta Open Source. (2026). *React Native documentation*. https://reactnative.dev/docs/getting-started

Microsoft. (2026). *Build mobile apps with .NET MAUI*. https://learn.microsoft.com/dotnet/maui/

Apple Inc. (2026). *Swift Documentation*. https://developer.apple.com/documentation/swift/

Mapa de fuentes por apartado

Apartado	Fuentes principales
1. Limitaciones	Android vitals · *Guide to Android app architecture* · Bluetooth LE Audio · fichas técnicas de los cinco fabricantes
2. Tecnologías	Documentación de Kotlin, Oracle, Flutter, React Native, Microsoft y Apple · blog Compose (19/05/2026)
3. Entorno	Android Studio · Install · Create project · Emulator · Hardware acceleration · AVDs · Troubleshooting · uses-sdk
4. Config. dispositivos	Screen densities · Screen sizes · Configuration.screenLayout · API levels · Target API · Storage · App Bundles · APK Analyzer · vitals · ActivityManager · 5G · Bluetooth · NFC · Wi-Fi · Wi-Fi 7 · Behavior changes
5. Perfiles	Make your app more accessible · Principles · Accessibility in Compose · TalkBack · WCAG 2.2 · Material Design 3 · Bluetooth LE Audio

Cuadro 35: Mapa de fuentes por apartado

⚠ Lo que queda pendiente de hacer tú (y no se puede hacer desde aquí).
Este documento está completo en su parte documental, pero **falta la parte práctica**, y esa la tienes que hacer tú en tu ordenador:

1. **Rellenar la portada.** Los tres campos marcados como *(pendiente de completar)*: **nombre**, **curso** y **fecha de entrega**.
2. **Instalar Android Studio** siguiendo el apartado 3.1 y hacer las **8 capturas** marcadas con `[CAPTURA PENDIENTE N]`. Cada captura necesita su **pie de imagen debajo**; las figuras 3.4 y 3.6 requieren además **versión ampliada**.
3. **Meter las capturas** en su sitio dentro del apartado 3 y **borrar los marcadores** `[CAPTURA PENDIENTE]`.
4. **Exportar a PDF.** Los números de página ya los incluye automáticamente este diseño.
5. **Revisar que las conclusiones estén redactadas con tus palabras.** El documento está en primera persona porque ese era el tono original; reescribe al menos introducción y conclusiones finales con tu estilo.
6. **Revisar tildes y puntuación.** El texto del apartado 1.1 es tuyo y está tal cual, con alguna errata (*temorales* por *temporales*, *aplicaion*, *milimentrica*, *puedelentizar*, *¿porque?*).

Cuando hayas hecho los puntos 1 a 4, el trabajo estará listo para entregar. El punto 5 es el que diferencia un trabajo aprobado de uno que se recuerda.